



Gustavo Teodoro Bauke

Interface Web para Regressão Simbólica

Santo André - SP

2025

Gustavo Teodoro Bauke

Interface Web para Regressão Simbólica

Universidade Federal do ABC
Curso de Ciência da Computação

Orientador: Prof. Dr. Fabrício Olivetti de França

Santo André - SP

2025

Dedicatória...

AGRADECIMENTOS

Agradecimentos...

Frase bonitinha sobre computação

RESUMO

RESUMO

Palavras-chave: Regressão Simbólica. Regressão. Aprendizado de Máquina. Interface Web.

ABSTRACT

ABSTRACT

Keywords: Symbolic Regression. Regression. Machine Learning. Web Interface.

SUMÁRIO

	Lista de ilustrações	9
	Lista de Códigos	10
1	INTRODUÇÃO	11
1.1	Contextualização	11
1.2	Objetivos	11
1.3	Justificativa	12
1.4	Organização deste trabalho	12
2	REVISÃO DA LITERATURA	13
2.1	Algoritmos de Regressão	13
2.2	Algoritmos Genéticos	13
2.2.1	Operador de seleção	14
2.2.2	Operador de crossover	14
2.2.3	Operador de mutação	14
2.3	Regressão Simbólica	15
2.3.1	Programação Genética para Regressão Simbólica	15
2.3.2	Saturação de igualdade para Regressão Simbólica	17
2.3.2.1	Grafos de Igualdade	18
2.3.2.2	Saturação de igualdade	19
2.3.2.3	Algoritmo: Eggp	19
2.3.3	Outros métodos para Regressão Simbólica	20
2.3.3.1	Interação-Transformação	20
2.3.3.2	Regressão Simbólica Exaustiva	22
2.4	Ferramentas para Regressão Simbólica	22
2.5	Conclusão	23
3	MATERIAIS E MÉTODOS	24
3.1	Metodologia	24
3.2	Ambiente de Desenvolvimento e Ferramentas	24
3.2.1	Desenvolvimento Front-end	24
3.2.2	Estrutura do Projeto	26
3.3	Arquitetura	28
3.3.1	Visão Geral e Fluxo de Dados	29
3.3.1.1	Front-End	29

3.3.1.2	Back-End	30
3.3.1.3	Workers	31
3.3.1.4	Armazenamento	31
3.3.2	Abstrações e Conceitos Base	34
3.3.2.1	Abstrações de Banco de Dados	34
3.3.3	Abstrações de Domínio	38
4	PLATAFORMA WEB PARA EXPLORAÇÃO DE ALGORITMOS DE REGRESSÃO SIMBÓLICA	45
4.1	Panorama Geral	45
4.2	Histórias de Usuário	45
4.3	Interface Web e Blocos de Visualização	46
4.4	Requisitos não-funcionais	47
4.5	Back-End	47
4.5.1	Usuários	48
4.5.2	Datasets	51
4.5.3	Profiles	51
4.6	Workers	51
4.7	Front-End	51
5	LINGUAGEM DE DOMÍNIO PARA EXPLORAÇÃO DE REGRESSÃO SIMBÓLICA	52
5.1	Contexto	52
5.2	Inference Query Language	52
5.2.1	Gramática Formal	53
5.2.2	Exemplos de Consultas	53
6	RESULTADOS E DISCUSSÃO	55
6.1	Estudo de Caso: Identificação de Leis Físicas	55
6.2	Análise de Performance	55
6.3	Discussão sobre Interpretabilidade	55
7	CONCLUSÃO	57
	Referências	58

LISTA DE ILUSTRAÇÕES

Figura 2 – Exemplo de uma árvore de sintaxe abstrata (AST) para a expressão $\log(x/2) + 3x$	15
Figura 3 – Exemplo de cruzamento entre duas expressões representadas por árvores de sintaxe abstrata (ASTs).	17
Figura 4 – Exemplo de mutação em uma expressão representada por uma árvore de sintaxe abstrata (AST).	17
Figura 5 – Exemplo de um grafo de igualdade, adaptado de (FRANÇA; KRONBERGER, 2025a).	18
Figura 6 – Exemplo de árvores geradas por regressão simbólica exaustiva.	22
Figura 7 – Diagrama de alto nível da arquitetura do projeto.	29
Figura 8 – Diagrama de alto nível da arquitetura do back-end.	30
Figura 9 – Diagrama de entidades do sistema	48

LISTA DE CÓDIGOS

3.1	Bibliotecas utilizadas no front-end e suas versões	25
3.2	Ferramentas utilizadas no back-end e suas versões	27
3.3	Definição da interface de armazenamento	32
3.4	Implementação da interface de armazenamento no back-end	32
3.5	Definição de uma entidade mutável em nível de banco de dados	35
3.6	Definição de uma entidade imutável em nível de banco de dados	35
3.7	Definição de uma entidade versionada em nível de banco de dados	36
3.8	Definição de uma entidade mutável em nível de domínio	38
3.9	Definição de uma entidade imutável em nível de domínio	38
3.10	Definição de uma entidade versionada em nível de domínio	38
3.11	Definição de um protocolo de repositório para entidades mutáveis	39
3.12	Definição de um protocolo de repositório para entidades imutáveis	40
3.13	Definição de um protocolo de repositório para entidades versionadas	40
3.14	Definição de um protocolo de repositório para operações em lote	41
3.15	Definição de um protocolo para o padrão de código Unit of Work	42
3.16	Definição de um protocolo para recursos transacionais	43
4.1	Código SQLAlchemy para criação da tabela de usuários	49
4.2	Classe de domínio que define um usuário	49
5.1	Estrutura básica de uma consulta IQL	53

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

Com evoluções tecnológicas constantes, o uso de técnicas de inteligência artificial (IA), principalmente algoritmos de aprendizado de máquina (AM), tem se tornado cada vez mais comum em diversas áreas (MAKKE; CHAWLA, 2024). Atualmente, as principais técnicas de IA utilizadas são baseadas em redes neurais e transformadores, que são capazes de aprender padrões complexos a partir de grandes volumes de dados. No entanto, essas técnicas muitas vezes resultam em modelos que são considerados "caixas-pretas", ou seja, cuja lógica interna é difícil de interpretar e compreender totalmente (FRANÇA; KRONBERGER, 2025a). Em algumas áreas, como na medicina, a interpretabilidade dos modelos é crucial, pois permite um entendimento mais profundo dos fenômenos capturados pelos dados.

Dessa forma, outras técnicas de IA, como a regressão simbólica, tem ganhado destaque como forma de obter modelos mais interpretáveis. A regressão simbólica é uma técnica que busca encontrar expressões matemáticas que melhor se ajustam aos dados observados e que respeitem um conjunto pré-determinado de operações matemáticas (FRANÇA; KRONBERGER, 2025a,b; BARTLETT; DESMOND; FERREIRA, 2024). Algoritmos de regressão simbólica são capazes de gerar modelos matemáticos que podem ser facilmente interpretados e analisados, o que os torna uma alternativa interessante para aplicações onde a transparência é importante. Várias técnicas podem ser utilizadas para implementar a regressão simbólica, incluindo algoritmos evolutivos ou técnicas híbridas que combinam redes neurais com algoritmos evolutivos.

1.2 OBJETIVOS

O objetivo deste trabalho é propor uma nova plataforma para manipulação de modelos de regressão simbólica. Para isso, primeiramente será feita uma revisão de literatura, apresentando as principais características, técnicas e aplicações da regressão simbólica, bem como comparação com outras técnicas de inferência. Dentre os objetivos específicos da plataforma proposta, podemos destacar: - Gerenciamento de pipelines de manipulação de dados - Exploração das soluções encontradas - Identificação de padrões presentes nas soluções encontradas - Seleção dos meta-parâmetros do algoritmo - Plotagem de gráficos com informações pertinentes sobre o conjunto de dados - Manipulação do conjunto de dados - Geração de datasets aleatórios para testes da ferramenta - Consulta condicional de soluções por meio de uma linguagem similar ao SQL

1.3 JUSTIFICATIVA

Atualmente, ferramentas como TuringBot e HeuristicLab podem ser utilizadas para criação de modelos de regressão simbólica. Apesar das vastas funcionalidades dessas ferramentas, elas possuem limitações em termos de acessibilidade e praticidade de uso, exigindo, muitas vezes, instalação local. Para máquinas fracas, executar os algoritmos de regressão simbólica pode se tornar um desafio, necessitando alternativas de execução remota. Ademais, as ferramentas existentes não permitem a criação de pipelines completas de importação, processamento e extração de dados. A interface web proposta neste trabalho visa superar essas limitações, oferecendo uma plataforma acessível, na qual o treinamento dos modelos é feito remotamente, permitindo que usuários possam explorar a regressão simbólica de forma mais intuitiva.

Além disso, essa plataforma permitirá que usuários explorem soluções por meio de uma linguagem de consulta, similar ao SQL, para selecionar padrões matemáticos de interesse dentro do espaço de soluções proposto pelo algoritmo, garantindo que conhecimentos prévios sobre os fenômenos estudados possam ser incorporados ao processo de modelagem.

A plataforma também permitirá a criação de pipelines completos para manipulação de conjuntos de dados de forma totalmente autônoma, permitindo a construção de processos complexos que dependem de modelos de regressão simbólica, podendo ser utilizada para mecanismos de previsão por exemplo.

1.4 ORGANIZAÇÃO DESTE TRABALHO

Este trabalho está organizado da seguinte forma: o capítulo 2 traz uma revisão da literatura atual sobre regressão simbólica, além de explorar os conceitos necessários para entendimento das principais implementações de algoritmos de regressão simbólica. O capítulo começa com uma revisão dos principais conceitos relacionados a regressão e algoritmos genéticos, depois o capítulo explica como algoritmos de regressão simbólica funcionam. Em seguida, o capítulo 2 discorre sobre as principais implementações de regressão simbólica atuais e, por fim, há uma breve comparação entre as principais ferramentas de regressão simbólica disponíveis atualmente. O capítulo 3 apresenta as metodologias utilizadas na construção desse projeto. Os próximos dois capítulos apresentam, respectivamente, a proposta de plataforma web para exploração de algoritmos de regressão simbólica e a linguagem de consulta proposta para essa exploração. O capítulo 6 apresenta os resultados obtidos com a implementação da plataforma e, por fim, o capítulo 7 traz as conclusões do trabalho e sugestões para trabalhos futuros.

2 REVISÃO DA LITERATURA

2.1 ALGORITMOS DE REGRESSÃO

Regressão é o processo de descobrir relações entre variáveis de entrada e de saída a partir de um conjunto de dados (STULP; SIGAUD, 2015). As técnicas de regressão podem ser divididas em duas categorias principais: regressão paramétrica e não paramétrica.

A regressão paramétrica fixa uma relação funcional entre as variáveis e seu objetivo é estimar os parâmetros dessa relação. Por outro lado, a regressão não paramétrica não assume uma forma funcional específica, permitindo maior flexibilidade na modelagem de dados complexos (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Algoritmos de regressão são comumente utilizados para interpolação e extrapolação de dados. Interpolação é o processo de estimar valores dentro do espaço de dados utilizados para o treinamento do modelo de regressão, enquanto extrapolação é o processo de estimar valores fora desse espaço. Diferentes algoritmos possuem propriedades diferentes em relação a suas capacidades de interpolação e extrapolação (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Existem diversas formas de implementar algoritmos de regressão, a depender do tipo de regressão desejado para o modelo. Este capítulo focará em algoritmos de regressão não paramétricos de ordem arbitrária, especificamente no algoritmo de Regressão Simbólica implementado via programação genética e grafos de igualdade. Além disso, serão abordadas comparações com outras técnicas de implementação de regressão simbólica.

2.2 ALGORITMOS GENÉTICOS

Algoritmos genéticos (AGs) são uma classe de algoritmos inspirados na evolução natural, onde soluções são evoluídas ao longo de gerações para resolver problemas complexos. A ideia principal do algoritmo é simular o processo de seleção natural, no qual a pressão exercida pelo ambiente seleciona os melhores indivíduos de uma espécie. Esses algoritmos utilizam operações como seleção, cruzamento e mutação para explorar o espaço de soluções (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024). O objetivo principal é otimizar um objetivo arbitrário em relação a um conjunto de variáveis de entrada (BEYER; SCHWEFEL, 2002). Algoritmos genéticos são amplamente utilizados em problemas de busca, otimização, decisão e aprendizado de máquina (LAMBORA; GUPTA; CHOPRA, 2019).

O funcionamento básico do algoritmo segue os seguintes passos:

1. Uma população inicial de soluções é gerada aleatoriamente;
2. A cada iteração, os indivíduos da população são avaliados com base em uma função de aptidão, que indica o quão bem cada indivíduo resolve o problema;
3. Os melhores indivíduos são selecionados para reprodução;
4. Operadores de cruzamento e mutação são aplicados para gerar novos indivíduos;
5. A nova população é formada com os indivíduos gerados;
6. O processo é repetido até que um critério de parada seja atingido.

Para implementação do algoritmo, é necessário definir a forma de representação dos indivíduos. Uma representação comumente utilizada é a codificação por cadeia de caracteres binária, na qual cada indivíduo é representada por uma sequência de bits, no qual cada bit representa uma propriedade do indivíduo (KATOCH; CHAUHAN; KUMAR, 2021). No algoritmo de regressão simbólica, abordado mais adiante, os indivíduos são representados por árvores de sintaxe abstrata, as quais codificam operações matemáticas.

2.2.1 OPERADOR DE SELEÇÃO

Seleção é o processo de escolher quais indivíduos da população atual serão utilizados para reprodução. Diversas estratégias de seleção podem ser utilizadas, como seleção por torneio e roleta. A seleção por torneio envolve a escolha dos melhores indivíduos de um subconjunto aleatório da população baseando-se nos maiores valores da função de aptidão. A seleção por roleta envolve a escolha de indivíduos com base em uma distribuição probabilística, onde indivíduos com maior aptidão têm maior probabilidade de serem selecionados (KATOCH; CHAUHAN; KUMAR, 2021).

2.2.2 OPERADOR DE CROSSOVER

Cruzamento é o processo de combinação dos genomas de dois indivíduos para geração de novos indivíduos (LAMBORA; GUPTA; CHOPRA, 2019; KATOCH; CHAUHAN; KUMAR, 2021). Normalmente, o cruzamento é realizado escolhendo-se um ponto aleatório de corte na representação genética dos indivíduos e trocando-se as partes dos genomas a partir desse ponto.

2.2.3 OPERADOR DE MUTAÇÃO

Mutação é o processo de alteração aleatória de um ou mais genes de um indivíduo para introduzir diversidade na população. O objetivo da mutação é evitar a convergência

prematura para solução sub-ótimas (LAMBORA; GUPTA; CHOPRA, 2019). Assim como no cruzamento, a mutação depende da forma utilizada para codificação dos indivíduos. Em uma representação em que os genes são expressos por uma sequência de bits, a mutação pode ser realizada invertendo-se um ou mais bits aleatoriamente.

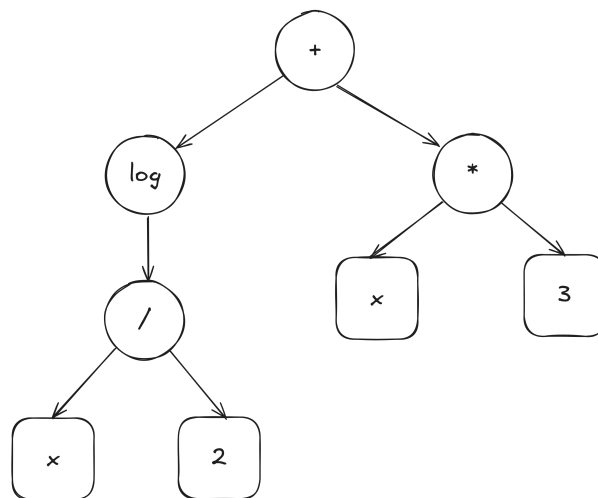
2.3 REGRESSÃO SIMBÓLICA

Regressão simbólica é uma técnica que busca encontrar expressões matemáticas que melhor descrevem a relação entre variáveis de entrada e saída em um conjunto de dados e pode ser categorizada como uma forma de regressão não paramétrica (FRANÇA; KRONBERGER, 2025a,b). Algoritmos considerados *State of the Art* para regressão simbólica utilizam algoritmos genéticos para evoluir expressões matemáticas que representam essas relações (OLIVETTI DE FRANÇA, 2018; KRONBERGER; BURLACU et al., 2024; FRANÇA; KRONBERGER, 2025a). No entanto, a regressão simbólica também pode ser implementada usando outros métodos, como redes neurais e transformadores (ANGELIS; SOFOS; KARAKASIDIS, 2023).

2.3.1 PROGRAMAÇÃO GENÉTICA PARA REGRESSÃO SIMBÓLICA

A programação genética utilizada na regressão simbólica evolui expressões matemáticas representadas por árvores de sintaxe abstrata (ASTs). Cada nó da árvore representa uma operação matemática (adição, subtração, multiplicação, exponenciação, etc.) e as folhas representam variáveis de entrada ou constantes. A Figura 2 ilustra um exemplo de árvore de sintaxe abstrata para a expressão $\log(x/2) + 3x$.

Figura 2 – Exemplo de uma árvore de sintaxe abstrata (AST) para a expressão $\log(x/2) + 3x$.



Nós que representam operações matemáticas são representados por círculos, enquanto nós que representam variáveis de entrada ou constantes são representados por

quadrados. Nós internos da árvore são chamados de não terminais, enquanto os nós folhas são chamados de terminais. O processo de busca envolve encontrar a melhor combinação de nós terminais e não terminais para minimizar a diferença entre os valores previstos pela expressão e os valores reais do conjunto de dados respeitando as restrições de complexidade e interpretabilidade da expressão (ANGELIS; SOFOS; KARAKASIDIS, 2023).

O processo de evolução se inicia com uma população inicial de expressões aleatórias. Em cada iteração, as expressões são avaliadas com base em uma função de aptidão, que mede o quão bem cada expressão se ajusta aos dados. As melhores expressões são selecionadas para reprodução, onde passam por operações de cruzamento e mutação para gerar novas expressões que são adicionadas à nova população. Esse processo continua até que um critério de parada seja atingido. Critérios de parada comuns incluem um número máximo de gerações, obtenção da expressão ideal ou convergência da população. Comumente, uma combinação entre todos os critérios é utilizada para garantir que o processo de evolução não seja interrompido prematuramente (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024).

A Figura 3 ilustra o processo de cruzamento entre duas expressões. Na parte superior, os indivíduos pais e, na parte inferior, os indivíduos produzidos por eles. Durante esse processo, subárvores aleatórias são selecionadas em cada expressão e trocadas entre si para criar dois novos indivíduos. Note que o cruzamento pode resultar em expressões mais complexas que os pais originais, o que pode ser controlado por meio de restrições de profundidade ou tamanho da árvore.

A Figura 4 ilustra o processo de mutação, onde uma subárvore aleatória é selecionada e substituída por uma nova subárvore gerada aleatoriamente. A mutação introduz diversidade na população, ajudando a evitar a convergência prematura para soluções sub-ótimas. Na esquerda, o indivíduo original, e na direita, o indivíduo após a mutação.

É importante destacar que esse algoritmo não precisa necessariamente retornar uma única expressão. Um conjunto de expressões pode ser retornado, permitindo ao usuário escolher a que melhor se adequa às suas necessidades (ANGELIS; SOFOS; KARAKASIDIS, 2023).

Apesar dos avanços recentes, a regressão simbólica ainda enfrenta vários desafios. A complexidade computacional decorrente do grande espaço de busca de expressões possíveis pode tornar o processo de evolução lento e ineficiente. Além disso, a interpretabilidade das expressões geradas pode ser comprometida quando elas se tornam excessivamente complexas, dificultando a compreensão por parte dos usuários finais (ANGELIS; SOFOS; KARAKASIDIS, 2023; KRONBERGER; BURLACU et al., 2024).

Além disso, a codificação de expressões como árvores pode levar o algoritmo de regressão a visitar expressões logicamente equivalentes várias vezes (FRANÇA; KRON-

Figura 3 – Exemplo de cruzamento entre duas expressões representadas por árvores de sintaxe abstrata (ASTs).

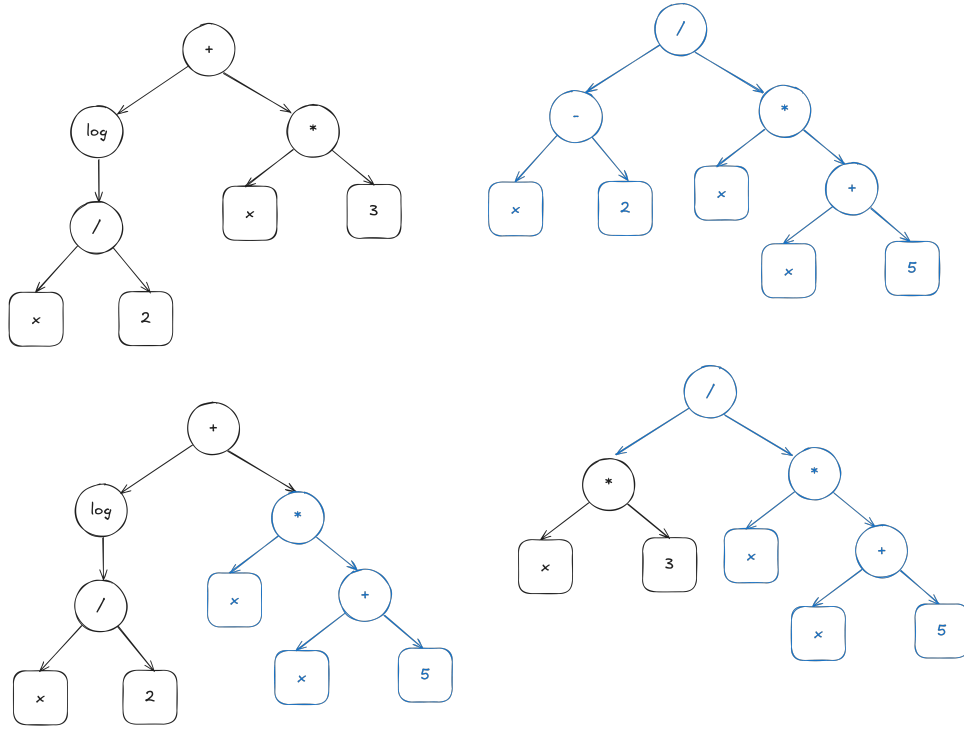
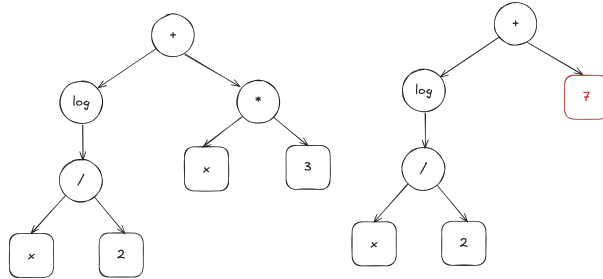


Figura 4 – Exemplo de mutação em uma expressão representada por uma árvore de sintaxe abstrata (AST).

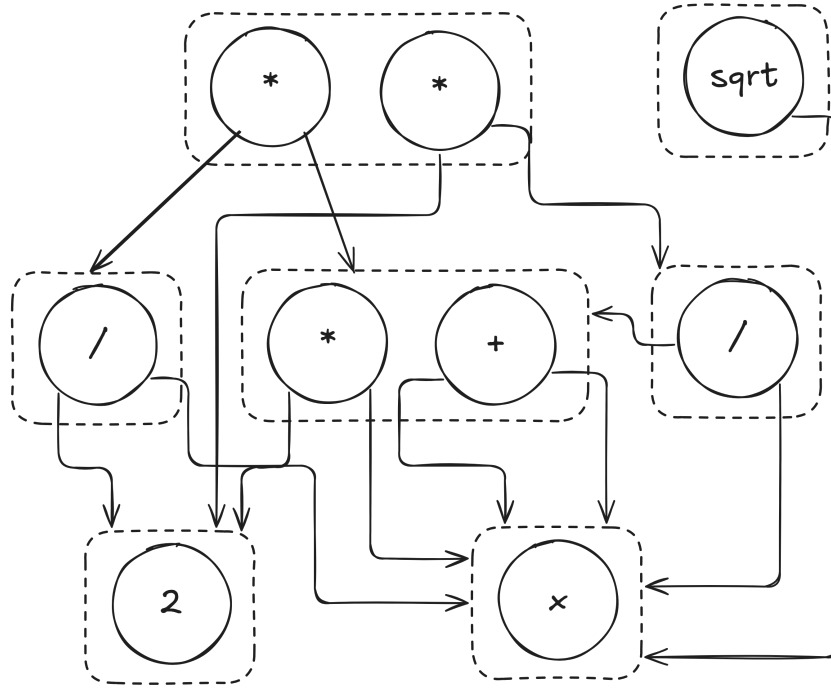


BERGER, 2025a). Por exemplo, as expressões $x + y$ e $y + x$ são logicamente iguais, mas seriam representadas por árvores diferentes. Testes empíricos indicam que apenas 40% das expressões geradas por algoritmos genéticos para regressão simbólica são únicas (KRONBERGER; OLIVETTI DE FRANÇA et al., 2024).

2.3.2 SATURAÇÃO DE IGUALDADE PARA REGRESSÃO SIMBÓLICA

Diante dos desafios enfrentados pela regressão simbólica tradicional, uma abordagem alternativa que tem ganhado destaque é o uso de grafos de igualdade (Equality Graphs - e-graphs) para representar e manipular expressões matemáticas (FRANÇA; KRONBERGER, 2025a,b). E-graphs são estruturas de dados que armazenam múltiplas expressões equivalentes de forma compacta, permitindo a aplicação eficiente de transformações algébricas e simplificações nos indivíduos presentes no espaço de busca. Utilizando essa

Figura 5 – Exemplo de um grafo de igualdade, adaptado de (FRANÇA; KRONBERGER, 2025a).



estrutura de dados, é possível realizar uma operação chamada de saturação de igualdade, que consiste em aplicar regras de reescrita para gerar todas as expressões logicamente equivalentes a uma dada expressão inicial.

2.3.2.1 GRAFOS DE IGUALDADE

Grafos de igualdade são estruturas de dados que representam expressões matemáticas e suas equivalências de maneira eficiente (FRANÇA; KRONBERGER, 2025a,b). Elementos matemáticos (operadores, variáveis e constantes) são representadas por e-nodes, enquanto classes de expressões equivalentes, ou seja, conjuntos de e-nodes equivalentes, são representadas por e-classes. Cada e-class contém um conjunto de e-nodes que representam diferentes formas de expressar a mesma expressão matemática. Cada e-node aponta para e-classes filhas, que representam os operandos da operação matemática representada pelo e-node, permitindo que variações de uma mesma expressão compartilhem subárvores e sejam representadas sem consumo elevado de memória.

A Figura 5 ilustra um grafo de igualdade. Note que as expressões $2x$ e $x + x$ pertencem à mesma e-class, uma vez que são logicamente equivalentes. A implementação de e-graphs utiliza uma estrutura union-find para gerenciar a união de e-classes quando novas equivalências são descobertas durante o processo de saturação (FRANÇA; KRONBERGER, 2025a). Uma e-class armazena uma lista de e-nodes que pertencem a ela, uma lista de e-nodes pais (e-nodes que possuem essa e-class como filha) e demais informações auxiliares para otimizar operações de busca e união.

2.3.2.2 SATURAÇÃO DE IGUALDADE

Saturação de igualdade é o processo de aplicar regras de reescrita a uma árvore para gerar expressões logicamente equivalentes. Quando uma equivalência entre expressões é encontrada, as e-classes correspondentes são fundidas, sinalizando que diferentes caminhos estruturais possuem mesmo significado semântico. O processo é eficiente porque todas as regras de reescrita são aplicadas paralelamente (FRANÇA; KRONBERGER, 2025a), utilizando a estrutura de e-graphs para evitar a duplicação de expressões. O processo de saturação acontece em quatro etapas:

1. Elenque todas as regras de equivalência para o estado atual do e-graph;
2. Aplique todas as regras;
3. Junte e-classes equivalentes;
4. Repita os passos anteriores até que nenhuma nova equivalência seja encontrada.

Após a aplicação repetida de todas as regras, o e-graph é considerado saturado. Nesse estado, todas as expressões equivalentes possíveis foram geradas e armazenadas no e-graph.

2.3.2.3 ALGORITMO: EGGP

Eggp (E-graph Genetic Programming) é uma implementação de um algoritmo genético para regressão simbólica que utiliza e-graphs e saturação de igualdade (FRANÇA; KRONBERGER, 2025a). O algoritmo segue os passos tradicionais de um algoritmo genético, mas com algumas diferenças importantes:

1. A cada nova expressão inserida no e-graph, é executada uma etapa de saturação, agrupando todas as expressões logicamente equivalentes;
2. Os operadores de mutação e cruzamento são adaptados para geração de expressões não visitadas;

Os operadores de mutação e cruzamento utilizam das informações do e-graph para gerar expressões ainda não visitadas. No cruzamento, uma subárvore aleatória de um dos pais é substituída por uma subárvore aleatória escolhida de um conjunto de todas as possíveis subárvores do outro pai. O conjunto é construído de tal forma que todos seus elementos, ao substituírem a subárvore original, geram expressões novas.

Na mutação, o processo tradicional é seguido, substituindo-se uma subárvore aleatória por uma nova subárvore gerada aleatoriamente. No entanto, caso a nova expressão já exista no e-graph, a subárvore é substituída por outra aleatória que faça parte do

conjunto de símbolos de mesma aridade da subárvore original. Da mesma forma que no cruzamento, esse conjunto é formado de tal maneira que todas as substituições geram expressões novas.

Dessa forma, a utilização de grafos de igualdade reduz drasticamente o espaço de busca da regressão simbólica, permitindo maior eficiência do algoritmo sem aumento significativo no custo computacional. Durante o processo de evolução, é comum que expressões semanticamente equivalentes sejam geradas e exploradas (FRANÇA; KRONBERGER, 2025a; KRONBERGER; OLIVETTI DE FRANÇA et al., 2024). A utilização de e-graphs permite que esses caminhos repetidos dentro do espaço de soluções sejam podados do processo de busca, otimizando o algoritmo.

2.3.3 OUTROS MÉTODOS PARA REGRESSÃO SIMBÓLICA

Apesar de algoritmos genéticos serem os mais comuns para implementação de regressão simbólica, outros métodos podem ser utilizados (BARTLETT; DESMOND; FERREIRA, 2024; MAKKE; CHAWLA, 2024): - Aprendizado por reforço; - Redes neurais; - Transformadores.

Para fins de comparação, esse trabalho também aborda uma implementação de regressão simbólica conhecida como Interação-Transformação (IMAI ALDEIA; FRANÇA, 2018), que utiliza uma abordagem diferente para geração de expressões matemáticas. Além disso, também será abordada uma técnica conhecida como Regressão Simbólica Exaustiva (BARTLETT; DESMOND; FERREIRA, 2024), que utiliza uma técnica similar à Interação-Transformação, mas que visa explorar todo o espaço de expressões possíveis dentro de um recorte delimitado, garantindo que a solução ótima seja encontrada.

2.3.3.1 INTERAÇÃO-TRANSFORMAÇÃO

A regressão simbólica utilizando da estrutura de Interação-Transformação (IT) é uma abordagem que restringe o espaço de busca de expressões matemáticas para aquelas que podem ser representadas como uma combinação polinomial das variáveis seguidas por uma transformação não-linear (IMAI ALDEIA; FRANÇA, 2018). Diferentemente dos algoritmos genéticos, nessa implementação, algumas relações não podem ser representadas.

Nessa representação, os termos são representados por uma tupla $(\mathbf{P}, \mathbf{t})$, onde $\mathbf{P}$ é um vetor de expoentes e $\mathbf{t}$ é uma função de transformação sendo aplicada sobre os termos do polinômio (IMAI ALDEIA; FRANÇA, 2018). Como um exemplo dessa representação, considere uma regressão de quatro variáveis de entrada, x_1 , x_2 , x_3 e x_4 . Considere que a função de transformação seja $t(x) = \log(x)$. Então, teríamos a seguinte representação:

$$\begin{aligned}
\mathbf{T} &= (x_1, x_2, x_3, x_4) \\
\mathbf{t}_1 &= ([1, 0, 0, 0], \log) \\
\mathbf{t}_2 &= ([0, 1, 0, 0], \log) \\
\mathbf{t}_3 &= ([0, 0, 1, 0], \log) \\
\mathbf{t}_4 &= ([0, 0, 0, 1], \log)
\end{aligned} \tag{2.1}$$

Para gerar novas expressões, o algoritmo utiliza uma abordagem de busca em largura. A cada iteração, o algoritmo gera todos os termos possíveis decorrentes da combinação dos termos já existentes de forma que $t_k = (P_i + P_j, id)$ e $t_l = (P_i - P_j, id)$. No exemplo anterior, teríamos para a combinação de t_1 e t_2 os seguintes termos:

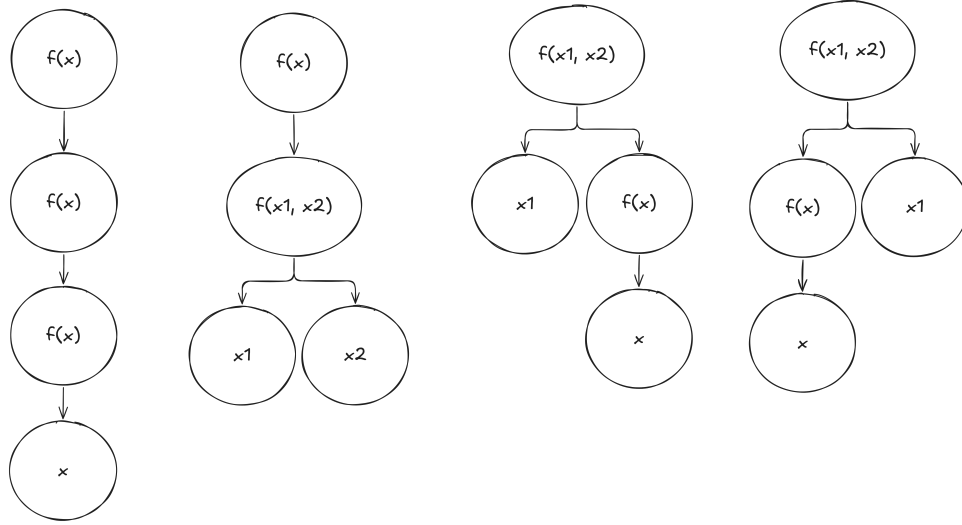
$$\begin{aligned}
\mathbf{t}_5 &= ([1, 1, 0, 0], \log) \\
\mathbf{t}_6 &= ([1, -1, 0, 0], \log)
\end{aligned} \tag{2.2}$$

Para etapa de transformação, para cada termo $(\mathbf{P}, t)$, o algoritmo gera um novo termo $(\mathbf{P}, t')$ para cada função de transformação $t' = t$ disponível. No exemplo anterior, considerando que as funções de transformação disponíveis sejam $\log$, $\sqrt{}$ e e^x , teríamos os seguintes termos:

$$\begin{aligned}
\mathbf{t}_7 &= ([1, 0, 0, 0], \sqrt{}) \\
\mathbf{t}_8 &= ([1, 0, 0, 0], e^x) \\
\mathbf{t}_9 &= ([0, 1, 0, 0], \sqrt{}) \\
\mathbf{t}_{10} &= ([0, 1, 0, 0], e^x) \\
\mathbf{t}_{11} &= ([0, 0, 1, 0], \sqrt{}) \\
\mathbf{t}_{12} &= ([0, 0, 1, 0], e^x) \\
\mathbf{t}_{13} &= ([0, 0, 0, 1], \sqrt{}) \\
\mathbf{t}_{14} &= ([0, 0, 0, 1], e^x) \\
\mathbf{t}_{15} &= ([1, 1, 0, 0], \sqrt{}) \\
\mathbf{t}_{16} &= ([1, 1, 0, 0], e^x) \\
\mathbf{t}_{17} &= ([1, -1, 0, 0], \sqrt{}) \\
\mathbf{t}_{18} &= ([1, -1, 0, 0], e^x)
\end{aligned} \tag{2.3}$$

Para cada expressão gerada, o algoritmo avalia a aptidão da expressão. Todas as expressões que não atingirem um valor mínimo de aptidão são descartadas (IMAI ALDEIA; FRANÇA, 2018).

Figura 6 – Exemplo de árvores geradas por regressão simbólica exaustiva.



2.3.3.2 REGRESSÃO SIMBÓLICA EXAUSTIVA

Semelhantemente a regressão simbólica utilizando Interação-Transformação, a regressão simbólica exaustiva busca limitar o espaço de busca, mas com o objetivo de explorar todo o espaço, garantindo que a solução ótima seja encontrada (BARTLETT; DESMOND; FERREIRA, 2024).

Nessa abordagem, a geração de expressões é feita por meio da geração de todas as combinações possíveis entre operadores de aridade zero, um e dois. A aridade zero é representada por constantes, a aridade um por funções unárias (como $\log$ e $\sqrt{}$) e a aridade dois por funções binárias (como $+$, $-$, $*$ e $/$) (BARTLETT; DESMOND; FERREIRA, 2024). Considere uma árvore de profundidade máxima $d = 4$ e um conjunto de operadores de aridade zero (x), um ($f(x)$) e dois ($f(x_1, x_2)$). Nesse caso, temos apenas quatro árvores válidas, conforme ilustrado na Figura 6.

Após a geração de todas as expressões, o algoritmo checa por expressões logicamente equivalentes, removendo-as do conjunto de expressões (BARTLETT; DESMOND; FERREIRA, 2024).

2.4 FERRAMENTAS PARA REGRESSÃO SIMBÓLICA

Existem diversas ferramentas disponíveis para exploração de algoritmos de regressão simbólica. Essas ferramentas variam desde implementações comerciais até bibliotecas de código aberto, cada uma com suas próprias características e funcionalidades.

HeuristicLab é uma ferramenta gráfica que, dentre outros algoritmos, suporta a exploração de algoritmos de regressão simbólica. Ele permite a criação de algoritmos personalizados e a visualização dos resultados de forma interativa. A ferramenta é extensível

e pode ser adaptada para diferentes necessidades de pesquisa (WAGNER et al., 2014). A ferramenta exibe a fronteira de Pareto final, detalhes estatísticos do modelo gerado, gráficos de desempenho e a representação final da árvore de busca.

Assim como a ferramenta anterior, TuringBot é uma interface gráfica para exploração de algoritmos de regressão simbólica. Ele mostra ao usuário as melhores expressões encontradas (fronteira de Pareto) para o conjunto de dados fornecido com algumas métricas simples do modelo criado (TURINGBOT SOFTWARE, 2020).

Para além de ferramentas comerciais, existem diversas bibliotecas que permitem a exploração de algoritmos de regressão simbólica. No caso do algoritmo Eggp, apresentado anteriormente, é possível utilizar a ferramenta *rEGGression* (FRANÇA; KRONBERGER, 2025b) para explorar os modelos gerados por esse algoritmo. Essa ferramenta permite que as expressões encontradas pelo modelo sejam consultadas por tamanho, complexidade e número de parâmetros numéricos; permite a inserção de novas expressões; permite a listagem de sub-expressões; e permite o uso de padrões para busca de expressões (FRANÇA; KRONBERGER, 2025b).

Apesar da existência de ferramentas que permitem exploração de modelos de regressão simbólica, poucas ferramentas permitem a criação de *pipelines* de dados para criação de modelos preditivos. As ferramentas disponíveis também não permitem a exploração de padrões dentro das expressões geradas.

2.5 CONCLUSÃO

A revisão de literatura apresentada neste capítulo retoma os principais conceitos e técnicas relacionados à regressão simbólica, incluindo algoritmos genéticos, programação genética e o uso de grafos de igualdade, dentre outras técnicas. Além disso, foram apresentadas ferramentas e implementações que permitem a exploração desses conceitos na prática. Nos próximos capítulos, será apresentada uma proposta de ambiente de exploração de algoritmos de regressão simbólica, com foco na implementação do algoritmo Eggp e na utilização de grafos de igualdade para otimização do processo de evolução de expressões matemáticas.

3 MATERIAIS E MÉTODOS

3.1 METODOLOGIA

O desenvolvimento deste projeto seguiu uma abordagem iterativa e incremental, baseando-se em metodologias ágeis em oposição ao modelo tradicional em cascata (*waterfall*). Enquanto o modelo cascata prevê uma sequência rígida de fases (requisitos, projeto, implementação, testes e manutenção), as metodologias ágeis permitem a adaptação contínua e a entrega frequente de valor ([SOMMERVILLE, 2011](#)).

Dentre as práticas ágeis, foram utilizados princípios do *Scrum*, como a divisão do trabalho em ciclos curtos e a revisão constante dos requisitos. Essa flexibilidade foi crucial para lidar com a complexidade da ferramenta, permitindo refinamento da solução a cada iteração. Além disso, a arquitetura do sistema foi guiada pelos princípios do *Domain-Driven Design* (DDD) ([EVANS, 2004](#)), garantindo que a lógica de negócio (o domínio da regressão simbólica) permanecesse isolada das complexidades de infraestrutura, como banco de dados e brokers de mensagens, o que permite a substituição de implementações específicas por outras desde que a interface definida pela aplicação seja respeitada.

3.2 AMBIENTE DE DESENVOLVIMENTO E FERRAMENTAS

O projeto foi desenvolvido utilizando o Windows Subsystem for Linux (WSL), uma ferramenta que permite a criação de máquinas virtuais Linux dentro de um computador Windows, e Docker, uma ferramenta que permite a containerização de aplicações.

O Docker foi utilizado para facilitar o gerenciamento do ambiente de desenvolvimento back-end, permitindo a criação de dois serviços essenciais para a aplicação: Postgres (Banco de Dados Relacional) e RabbitMQ (Broker de Mensagens). O uso dessa ferramenta permite a replicabilidade do ambiente de desenvolvimento em qualquer máquina, facilitando o processo de criação e execução do código.

3.2.1 DESENVOLVIMENTO FRONT-END

A construção das telas da plataforma e suas funcionalidades foi feita através do *framework* React (desenvolvido e mantido pela Meta) que utiliza a linguagem imperativa *JavaScript*. Além disso, as bibliotecas *Tanstack React Query* e *Zustand* foram utilizadas, respectivamente, para gerenciamento de estado assíncrono, ou seja, informações vindas diretamente do back-end da aplicação, e para gerenciamento do estado interno da aplicação front-end.

Demais ferramentas utilizadas e suas versões específicas podem ser encontradas abaixo:

Código 3.1 – Bibliotecas utilizadas no front-end e suas versões

```
1 "dependencies": {
2   "@hookform/resolvers": "^5.2.2",
3   "@monaco-editor/react": "^4.8.0-rc.3",
4   "@react-router/node": "^7.9.2",
5   "@react-router/serve": "^7.9.2",
6   "@tanstack/react-query": "^5.90.21",
7   "@tanstack/react-query-devtools": "^5.91.3",
8   "@tanstack/react-table": "^8.21.3",
9   "clsx": "^2.1.1",
10  "framer-motion": "^12.35.2",
11  "isbot": "^5.1.31",
12  "lucide-react": "^0.577.0",
13  "papaparse": "^5.5.3",
14  "react": "^19.1.1",
15  "react-dom": "^19.1.1",
16  "react-dropzone": "^14.3.8",
17  "react-hook-form": "^7.69.0",
18  "react-intersection-observer": "^10.0.3",
19  "react-katex": "^3.1.0",
20  "react-markdown": "^10.1.0",
21  "react-router": "^7.9.2",
22  "rehype-highlight": "^7.0.2",
23  "remark-gfm": "^4.0.1",
24  "tailwind-merge": "^3.4.0",
25  "uuid": "^13.0.0",
26  "zod": "^4.3.6",
27  "zustand": "^5.0.11"
28 },
29 "devDependencies": {
30   "@react-router/dev": "^7.9.2",
31   "@tailwindcss/typography": "^0.5.19",
32   "@tailwindcss/vite": "^4.1.13",
33   "@types/node": "^22",
34   "@types/papaparse": "^5.5.2",
35   "@types/react": "^19.1.13",
36   "@types/react-dom": "^19.1.9",
```

```

37     "@types/react-katex": "^3.0.4",
38     "tailwindcss": "^4.1.13",
39     "typescript": "^5.9.2",
40     "vite": "^7.1.7",
41     "vite-tsconfig-paths": "^5.1.4"
42 }

```

Para permitir o treinamento de modelos de regressão de forma assíncrona, evitando o bloqueio do servidor HTTP, o sistema utiliza *workers* independentes em Python que se comunicam com a API principal através do *RabbitMQ*, integrado via biblioteca *aio_pika*. Por fim, as operações de regressão simbólica foram feitas utilizando as bibliotecas *eggp* para o treinamento, e *rEGGression* para manipulação dos modelos. O *PostgreSQL* foi o banco de dados escolhido devido à natureza fortemente estruturada e relacional das entidades do sistema.

3.2.2 ESTRUTURA DO PROJETO

A organização do código-fonte segue uma estrutura modular para facilitar a manutenção e o desacoplamento entre os componentes. Abaixo é apresentada a árvore de diretórios principal:

```

spectrum/
├── docker/ ..... Configurações de containers (Postgres, RabbitMQ)
├── packages/
│   ├── backend/ ..... API FastAPI e lógica de aplicação
│   ├── core/ ..... Domínio puro, entidades e interfaces (ports)
│   ├── db_core/ ..... Implementação de repositórios e SQLAlchemy
│   ├── frontend/ ..... Interface React e gerenciamento de estado
│   ├── inference_query_language/ ..... Parser e executor da IQL
│   └── workers/ ..... Processamento especializado assíncrono
├── paper/ ..... Código-fonte LaTeX da monografia
└── scripts/ ..... Utilitários de desenvolvimento e automação

```

Cada pacote possui uma responsabilidade bem definida: o *core* isola a lógica de negócio da aplicação, definindo as entidades principais do projeto e as interfaces necessárias para manipulação dessas entidades; o *db_core* lida exclusivamente com a persistência, definindo as tabelas do banco de dados, bem como as interfaces responsáveis pela manipulação dos dados por meio de operações intermediadas pelo banco de dados; o *backend* orquestra as requisições HTTP e a comunicação com os *workers* via *RabbitMQ*; o *frontend* provê a camada de interação com o usuário final; e o *inference_query_language* isola a lógica da linguagem de domínio IQL, utilizada para interagir com os modelos de regressão

via sintaxe similar ao SQL.

Demais ferramentas utilizadas e suas versões específicas podem ser encontradas abaixo:

Código 3.2 – Ferramentas utilizadas no back-end e suas versões

```
// Servidor
dependencies = [
    "aio-pika>=9.5.8",
    "aiofiles>=25.1.0",
    "alembic>=1.18.3",
    "asyncpg>=0.31.0",
    "core",
    "db-core",
    "dotenv>=0.9.9",
    "eggpy>=1.0.16",
    "fastapi[standard]>=0.128.0",
    "inference-query-language",
    "pandas>=3.0.0",
    "pika>=1.3.2",
    "pydantic[email]>=2.12.5",
    "pydantic-settings>=2.12.0",
    "python-json-logger>=4.0.0",
    "regression>=1.0.10",
    "sqlalchemy>=2.0.46",
    "uvicorn[standard]",
    "argon2-cffi>=25.1.0",
    "jose>=1.0.0",
    "python-jose[cryptography]>=3.5.0",
]

[tool.uv]
dev-dependencies = [
    "pytest",
    "pytest-asyncio",
    "httpx",
    "ruff",
    "mypy",
    "pre-commit",
]

// Pacote Core
dependencies = [
```

```
"inference-query-language",
"pydantic>=2.12.5",
"pydantic-settings>=2.12.0",
"regression>=1.0.10",
]

// Pacote DB Core
dependencies = [
    "core",
]

// Pacote IQL
dependencies = [
    "pandas>=3.0.0",
    "pydantic>=2.12.5",
]

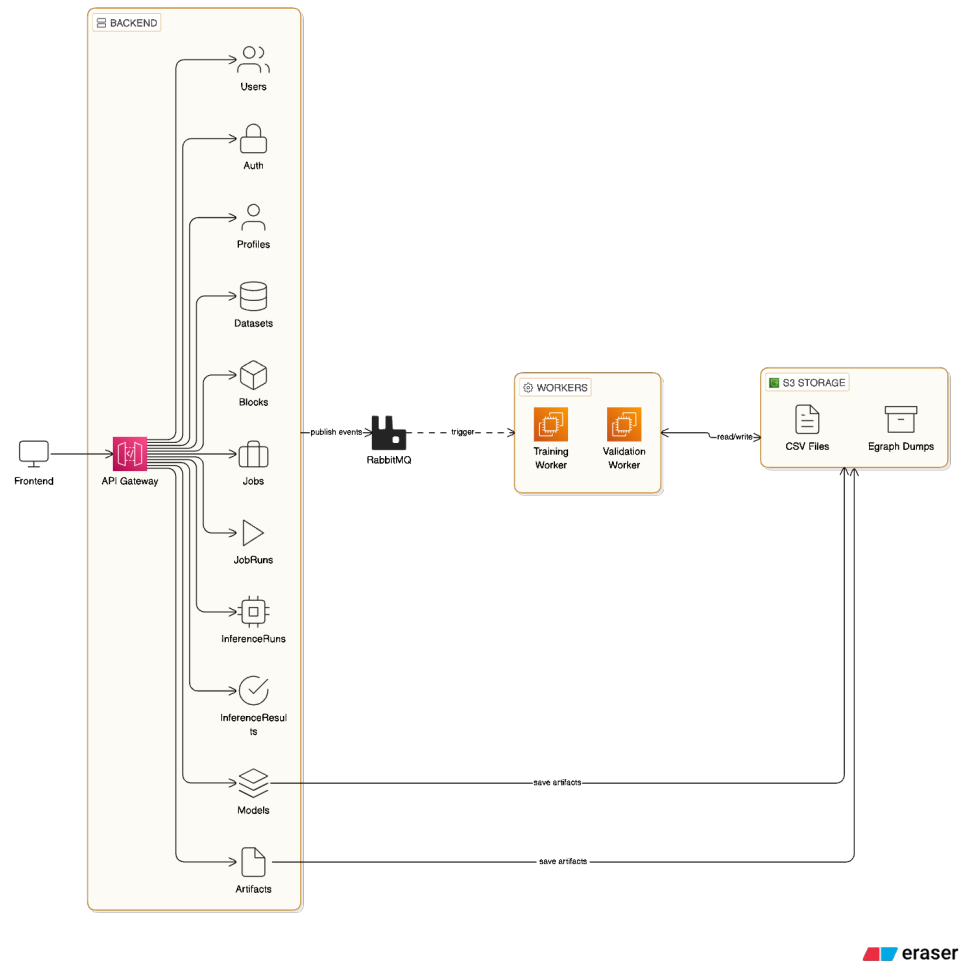
// Pacote Workers
dependencies = [
    "eggp>=1.0.16",
    "core",
    "db-core",
    "aio-pika>=9.5.8",
]
```

3.3 ARQUITETURA

A arquitetura da plataforma foi projetada sob o modelo cliente-servidor ([SOMMERVILLE, 2011](#)) com o objetivo de oferecer desacoplamento entre a interface gráfica (front-end) utilizada pelo usuário e a lógica de processamento, ou regras de negócio, e persistência de dados (back-end), permitindo a evolução independente de ambos os sistemas.

Nesta seção, é apresentado o fluxo geral de integração dos componentes do sistema, detalhando os padrões de projeto adotados, baseando-se nos princípios do *Domain Driven Design* ([EVANS, 2004](#)), uma forma de desenvolvimento de *software* que preza pelo desenvolvimento dentro de domínios específicos, ou seja, dentro de micro-universos que representam as regras de negócio da aplicação. No capítulo seguinte será apresentado os detalhes de implementação da plataforma, bem como suas principais funcionalidades e também pontos de melhoria e exploração futuros.

Figura 7 – Diagrama de alto nível da arquitetura do projeto.



3.3.1 VISÃO GERAL E FLUXO DE DADOS

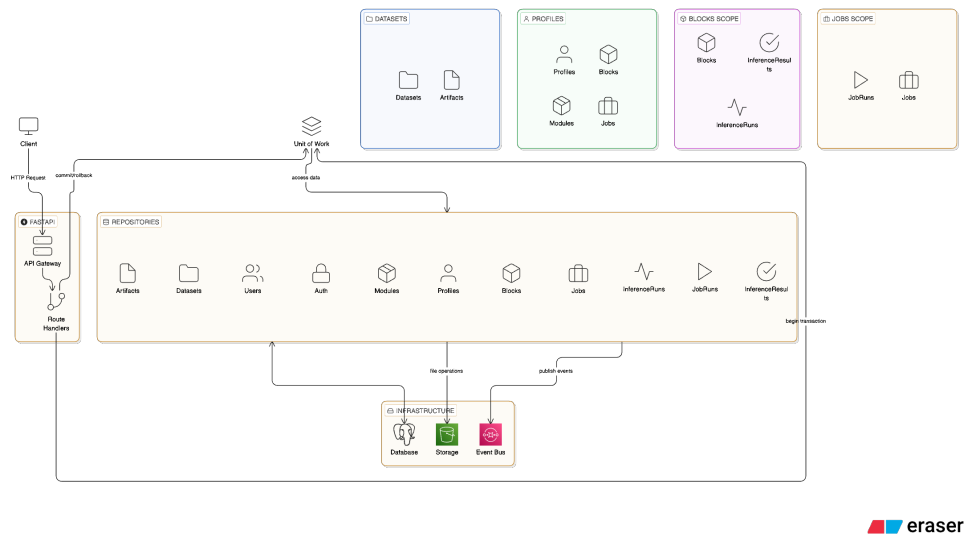
O diagrama 7 apresenta a arquitetura de alto nível do projeto, dividida em quatro principais componentes:

1. Front-end
2. Back-end
3. Workers
4. Armazenamento

3.3.1.1 FRONT-END

O front-end é responsável por mostrar ao usuário os dados vindos da API (*Application Programming Interface*) e permitir a manipulação desses dados. Por meio da interface, o usuário pode:

Figura 8 – Diagrama de alto nível da arquitetura do back-end.



1. Fazer upload de conjunto de dados
2. Criar *Profiles*, ou seja, espaços nos quais os conjuntos de dados podem ser importados para treinamento e manipulação de modelos
3. Criar modelos de regressão simbólica para os conjuntos de dados de uma *Profile*
4. Gerenciar (atualizar e/ou excluir) conjuntos de dados e *Profiles*
5. Criar uma conta na plataforma
6. Fazer login na conta criada
7. Escrever scripts IQL para manipulação dos modelos criados
8. Observar, por meio de elementos visuais como gráficos, propriedades dos modelos criados

Toda comunicação com o back-end é feita de forma assíncrona, por meio de requisições HTTP ou *Server Side Events (SSE)*, os quais permitem o back-end se comunicar diretamente com o front-end enviando dados via *Streaming*.

3.3.1.2 BACK-END

O back-end é responsável pelas regras de negócio da aplicação. Ele foi desenvolvido de forma modular, ou seja, cada módulo apresenta suas próprias regras de negócio e interfaces de comunicação próprias. É de responsabilidade do back-end também a comunicação com os demais serviços da plataforma, como os serviços de armazenamento (local ou S3, por exemplo), brokers de mensagens e *workers*.

O diagrama 8 mostra o fluxo de uma requisição HTTP na aplicação, bem como a divisão de escopos para as entidades presentes. O principal padrão estrutural do back-end é o *Unit of Work*, o qual possibilita retornar o estado da aplicação ao estado inicial caso um fluxo de mudanças encontre um erro. Esse padrão se utiliza de transações de banco de dados para garantir integridade do sistema (PROCURAR CITAÇÃO). As transações são controladas implicitamente pelo fluxo da aplicação Python, sendo automaticamente realizada a operação de *commit* assim que o escopo da função responsável por determinada funcionalidade se encerra, ou sendo realizado o *rollback* caso uma exceção seja levantada em código. Toda manipulação que necessita do banco de dados é realizada por meio de um repositório, o qual é uma classe que define operações base para cada entidade do sistema.

3.3.1.3 WORKERS

Os *workers* são responsáveis por treinar os modelos de regressão utilizando a biblioteca *eggpy* (FRANÇA; KRONBERGER, 2025a), e por validar os modelos criados, ou seja, calcular a fronteira de pareto do modelo e calcular os valores gerados por cada expressão de pareto. Assim como o back-end, as manipulações ao banco de dados são feitas via repositórios. O padrão *Unit of Work* também é utilizado, mas, diferentemente do back-end, o controle das transações é feito de forma manual.

Cada *worker* consome eventos específicos da fila de mensagens de forma assíncrona. Como cada *worker* atualiza as informações pertinentes para seu domínio, não há comunicação no sentido dos *workers* para o *back-end*. No entanto, *workers* podem se comunicar com a fila de mensagens, emitindo novos eventos para si mesmo ou outros *workers*.

3.3.1.4 ARMAZENAMENTO

O componente de armazenamento é responsável por armazenar os dados que não podem ser salvos no banco de dados, ou seja, artefatos necessários para o funcionamento da aplicação que não podem ser armazenados de forma tabular e relacional. Os principais elementos guardados nesse componente são:

1. Arquivos CSV para os conjuntos de dados que cada usuário fez upload
2. Arquivos para os e-graphs gerados pelo treinamento dos modelos
3. Arquivos CSV gerados pelo treinamento dos modelos

O componente de armazenamento apresenta dois modos de operação: modo local e modo remoto. No modo local, os arquivos são armazenados no sistema de arquivos do servidor que hospeda a aplicação. No modo remoto, os arquivos são armazenados em um S3, serviço da AWS para armazenar arquivos.

A interface de armazenamento é definida de forma abstrata, por meio de protocolos, no pacote *core*:

Código 3.3 – Definição da interface de armazenamento

```
from typing import Protocol , BinaryIO
from .upload_result import UploadResult

class FileStorage(Protocol):
    async def upload(
        self ,
        *,
        path: str ,
        file : BinaryIO ,
    ) -> UploadResult: ...

    async def download(
        self ,
        *,
        path: str ,
        destination: str ,
    ) -> None: ...
```

Uma implementação para essa interface deve definir os métodos *upload* e *download*. Cada componente da aplicação que acessa o componente de armazenamento define sua própria implementação, permitindo que detalhes de implementação sejam escondidos do restante da aplicação que necessita utilizar esse componente. Esse padrão é chamado de *Ports and Adapters* e permite a troca de implementações de serviços específicos dentro da aplicação desde que os métodos necessários sejam oferecidos por cada implementação. Abaixo, segue exemplo de implementação do serviço de armazenamento para o *back-end*:

Código 3.4 – Implementação da interface de armazenamento no back-end

```
import hashlib

from typing import BinaryIO
from pathlib import Path

from app.core.config import settings

from core.ports.storage.file_storage import FileStorage
from core.ports.storage.path import SpectrumPath
```

```

from core.ports.storage.upload_result import UploadResult

class LocalStorage(FileStorage):
    def __init__(self) -> None:
        self._base_path = Path(
            settings.FILE_STORAGE_SPECTRUM_DATA_PATH
        )

    async def upload(
        self,
        *,
        path: str,
        file: BinaryIO
    ) -> UploadResult:
        full_path = self._base_path / path
        full_path.parent.mkdir(parents=True, exist_ok=True)

        hasher = hashlib.sha256()
        size = 0

        file.seek(0)

        with open(full_path, 'wb') as f:
            while chunk := file.read(8192):
                f.write(chunk)
                hasher.update(chunk)
                size += len(chunk)

        spectrum_path = SpectrumPath(
            full_path=full_path, path=Path(path))
        checksum = hasher.hexdigest()

        return UploadResult(
            path=spectrum_path,
            size=size,
            checksum=checksum
        )

```

```

async def download(
    self,
    *,
    path: str,
    destination: str
) -> None:
    source_path = self._base_path / path
    destination_path = Path(destination)

    if not source_path.exists():
        raise FileNotFoundError(
            f"File {source_path} not found at {destination_path}")

    destination_path.parent.mkdir(
        parents=True, exist_ok=True)
    destination_path.write_bytes(source_path.read_bytes())

```

3.3.2 ABSTRAÇÕES E CONCEITOS BASE

Os pacotes **core** e **db_core** definem as principais abstrações utilizadas pelo sistema como forma de simplificar implementações e definir interfaces de comunicação claras e precisas que permitam a substituição de subsistemas caso necessário, isolando subsistemas do mundo exterior, de acordo com o padrão de código *Ports and Adapters* (MARTIN, 2008).

3.3.2.1 ABSTRAÇÕES DE BANCO DE DADOS

O pacote **db_core** utiliza as bibliotecas *SQLAlchemy* e *Alembic* para gerenciar as entidades do banco de dados. Para facilitar a implementação das entidades, foram definidas algumas classes utilitárias para padrões comuns:

1. **Entidades Mutáveis:** Entidades que podem ser alteradas de forma direta pelo usuário da aplicação. Suportam operações de escrita, leitura e deleção (CRUD). Definido pelo código 3.5. Toda entidade mutável apresenta as propriedades *id*, um identificador único gerado pelo padrão UUIDv4; *created_at*, um marcador temporal de quando a entidade foi criada no sistema; e *updated_at*, um marcador temporal de quando a entidade foi alterada no sistema;
2. **Entidades Imutáveis:** Entidades que não podem ser alteradas diretamente pelo usuário, podendo ser alteradas apenas pela própria plataforma Spectrum. Suportam apenas operações de criação e leitura. Definido pelo código 3.6;

3. **Entidades Versionadas:** Entidades que são imutáveis mas apresentam versões diferentes. Versões antigas são mantidas para consulta. Suportam apenas operações de criação e leitura. Definida pelo código [3.7](#)

Código 3.5 – Definição de uma entidade mutável em nível de banco de dados

```

from uuid import UUID
from datetime import datetime

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import func, DateTime, text
from sqlalchemy.dialects.postgresql import UUID as PG_UUID

from .root import RootBase

class MutableBase(RootBase):
    __abstract__ = True

    id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        primary_key=True,
        server_default=text("gen_random_uuid()"),
    )

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
        server_onupdate=func.now(),
    )

```

Código 3.6 – Definição de uma entidade imutável em nível de banco de dados

```

from typing import Any

```

```

from uuid import UUID
from datetime import datetime

from sqlalchemy.orm import Mapped, mapped_column, declared_attr
from sqlalchemy import func, DateTime, UniqueConstraint, Index,
    Boolean, text
from sqlalchemy.dialects.postgresql import UUID as PG_UUID

from .root import RootBase

class ImmutableBase(RootBase):
    __abstract__ = True

    id: Mapped[UUID] = mapped_column(
        PG_UUID(as_uuid=True),
        primary_key=True,
        server_default=text("gen_random_uuid()"),
    )

    timestamp: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        nullable=False,
        server_default=func.now(),
    )

    @declared_attr.directive
    def __table_args__(cls) -> Any:
        return (
            UniqueConstraint("id", name=f"uq_{cls.__tablename__}"
                               "_id"),
            Index(f"idx_{cls.__tablename__}_timestamp", "
                  timestamp"),
        )

```

Código 3.7 – Definição de uma entidade versionada em nível de banco de dados

```

class ImmutableVersionedBase(ImmutableBase):
    __abstract__ = True

```

```

id: Mapped[UUID] = mapped_column(
    PG_UUID(as_uuid=True),
    primary_key=True,
    server_default=text("gen_random_uuid()"),
)

version: Mapped[int] = mapped_column(
    nullable=False,
    primary_key=True,
    default=1,
)

is_latest: Mapped[bool] = mapped_column(
    Boolean,
    default=True,
    nullable=False,
)

timestamp: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    nullable=False,
    server_default=func.now(),
)

@declared_attr.directive
def __table_args__(cls) -> Any:
    return (
        UniqueConstraint(
            "id", "version", name=f"uq_{cls.__tablename__}_id_version"),
        Index(f"idx_{cls.__tablename__}_id_version", "id", "version"),
        Index(f"idx_{cls.__tablename__}_is_latest", "is_latest"),
        Index(f"idx_{cls.__tablename__}_latest", "id",
            unique=True, postgresql_where=text("is_latest = true"))
    )

```

3.3.3 ABSTRAÇÕES DE DOMÍNIO

O pacote **core** define as entidades de domínio do sistema, bem como interfaces comuns aos demais pacotes. As representações de domínio seguem uma estrutura similar àquelas definidas para o banco de dados. No entanto, as entidades de domínio não possuem detalhes de implementação relacionados à persistência, como anotações de tipo específicos para o banco de dados, índices ou restrições de chave estrangeira. Além disso, segundo as práticas do Código Limpo (MARTIN, 2008), entidades de domínio não precisam equivaler-se diretamente às entidades de banco de dados, podendo apresentar uma estrutura diferente, desde que as regras de negócio sejam respeitadas.

Assim como as entidades de banco de dados, as entidades de domínio também são divididas em mutáveis (código 3.8), imutáveis (código 3.9) e versionadas (código 3.10). Essas entidades são implementadas utilizando-se a biblioteca *Pydantic*, a qual permite a definição de modelos de dados com validação de dados acoplada, bem como facilita a conversão entre diferentes representações de dados, como dicionários, JSON e objetos Python.

Código 3.8 – Definição de uma entidade mutável em nível de domínio

```
class BaseMutableDomainModel(RootDomainModel):
    id: UUID = Field(default_factory=uuid4)
    created_at: datetime = Field(default_factory=lambda:
        datetime.now(timezone.utc))
    updated_at: datetime = Field(default_factory=lambda:
        datetime.now(timezone.utc))
```

Código 3.9 – Definição de uma entidade imutável em nível de domínio

```
class BaseImmutableDomainModel(RootDomainModel):
    id: UUID = Field(default_factory=uuid4)
    timestamp: datetime = Field(default_factory=lambda: datetime
        .now(timezone.utc))
```

Código 3.10 – Definição de uma entidade versionada em nível de domínio

```
class BaseImmutableVersionedDomainModel(RootDomainModel):
    id: UUID = Field(default_factory=uuid4)
    version: int = Field(default=1, description="Version number for optimistic concurrency control")
    timestamp: datetime = Field(default_factory=lambda: datetime
        .now(timezone.utc))
    is_latest: bool = Field(default=True)
```

Para além das definições bases para entidades de domínio, o pacote **core** também define os protocolos para os repositórios. Segundo as práticas do Código Limpo (MARTIN, 2008), repositórios são classes responsáveis por encapsular a lógica necessária para acessar fontes de dados, como bancos de dados, serviços externos ou mesmo arquivos locais, isolando o restante da aplicação dos detalhes de implementação relacionados à persistência de dados. As implementações específicas dos protocolos estão presentes no pacote **db_core**.

A definição dos protocolos de repositórios segue uma estrutura similar àquela das entidades de domínio, apresentando uma divisão entre repositórios para entidades mutáveis (código 3.11), imutáveis (código 3.12) e versionadas (código 3.13). Além disso, também é definido um protocolo para repositórios que permitem operações em lote (código 3.14), ou seja, operações que manipulam múltiplas entidades de forma simultânea.

Para garantir a segurança de tipos em tempo de compilação, os protocolos são implementados utilizando-se da funcionalidade de *Generics* do Python, permitindo a definição de tipos genéricos para as entidades manipuladas pelos repositórios, bem como para os filtros e cláusulas de busca utilizadas nas operações de leitura. Dessa forma, é possível garantir que as implementações específicas dos repositórios respeitem as interfaces definidas pelos protocolos, evitando erros de tipo em tempo de execução.

Código 3.11 – Definição de um protocolo de repositório para entidades mutáveis

```
from typing import Protocol

from core.features.base import (
    BaseMutableDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .immutable import ImmutableRepository

class ImmutableRepository[
    T_Mutable: BaseMutableDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](ImmutableRepository[T_Mutable, T_Where, T_Filter], Protocol):
    async def update(self, entity: T_Mutable) -> None: ...
    async def delete(self, entity: T_Mutable) -> None: ...
```


Código 3.12 – Definição de um protocolo de repositório para entidades imutáveis

```
from typing import Optional, Protocol, Sequence

from core.features.base import (
    RootDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter
from core.utils.pagination.response import PaginatedResponse
from core.utils.pagination.base import Pagination

class ImmutableRepository[
    T_Immutable: RootDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](Protocol):
    async def add(self, entity: T_Immutable) -> None: ...
    async def get_unique(self, where: T_Where) -> Optional[
        T_Immutable]: ...

    async def list(self, filter: Optional[T_Filter] = None,
                  pagination: Optional[Pagination] = None) ->
        PaginatedResponse[T_Immutable]: ...

    async def list_all(
        self, filter: Optional[T_Filter] = None) -> Sequence[
        T_Immutable]: ...
```

Código 3.13 – Definição de um protocolo de repositório para entidades versionadas

```
from typing import Optional, Protocol
from uuid import UUID

from core.features.base import BaseImmutableVersionedDomainModel
from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .immutable import ImmutableRepository
```

```

class IVersionedRepository [
    T_Versioned: BaseImmutableVersionedDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](ImmutableRepository[T_Versioned, T_Where, T_Filter], Protocol
):
    async def get_latest_version(
        self, parent_id: UUID) -> Optional[T_Versioned]: ...

    async def get_version_by_number(
        self, parent_id: UUID, version: int) -> Optional[
            T_Versioned]: ...

    async def unset_latest_and_add(
        self, parent_id: UUID, new_entity: T_Versioned) -> None:
        ...

```

Código 3.14 – Definição de um protocolo de repositório para operações em lote

```

from typing import Protocol, Iterable

from core.features.base import (
    BaseMutableDomainModel,
)

from core.utils.where import BaseUniqueWhere
from core.utils.filters.base import BaseFilter

from .mutable import ImmutableRepository

class ImmutableBulkRepository [
    T_Mutable: BaseMutableDomainModel,
    T_Where: BaseUniqueWhere,
    T_Filter: BaseFilter,
](ImmutableRepository[T_Mutable, T_Where, T_Filter], Protocol):
    async def add_many(self, entities: Iterable[T_Mutable]) ->
        None: ...

```

```

async def update_many(self, entities: Iterable[T_Mutable])
    -> None: ...
async def delete_many(self, entities: Iterable[T_Mutable])
    -> None: ...

```

Por fim, o pacote **core** também define protocolos para o padrão de código *Unit of Work*, definindo quais componentes podem ser controlados dentro de uma determinada unidade de trabalho. O padrão de código *Unit of Work* é um padrão de design que tem como objetivo manter um registro de todas as operações realizadas durante uma transação, garantindo que todas as operações sejam concluídas com sucesso ou revertidas em caso de falha, mantendo a integridade dos dados (MARTIN, 2008). Em conjunto com a definição do protocolo de unidades de trabalho, a plataforma também trabalha com o conceito de recursos transacionais, ou seja, recursos que precisam ser controlados dentro de uma unidade de trabalho para garantir a integridade do sistema, mas que não são associados a transações de banco de dados. Os principais recursos transacionais utilizados são conexões com brokers de mensagens e conexões com serviços de armazenamento de dados.

As definições para o protocolo de unidade de trabalho podem ser encontrados no código 3.15, enquanto as definições para o protocolo de recurso transacional podem ser encontradas no código 3.16.

Código 3.15 – Definição de um protocolo para o padrão de código Unit of Work

```

class UnitOfWork(ABC):
    users: IUsersRepository
    auth: IAuthRepository
    datasets: IDatasetsRepository
    artifacts: IArtifactsRepository
    profiles: IProfilesRepository
    blocks: IBlocksRepository
    jobs: IJobsRepository
    models: IModelsRepository
    runs: IRunsRepository
    inference_runs: IIInferenceRunRepository
    inference_results: IIInferenceResultRepository

    file_storage: FileStorage
    events_publisher: EventPublisher

    def __init__(self) -> None:
        self._resources: list[TransactionalResource] = []
        self._on_commit_hooks: list[Callable[[]], Awaitable[None]

```

```

    ]]] = []

    def register(self, resource: TransactionalResource) -> None:
        self._resources.append(resource)

    async def commit_resources(self) -> None:
        for resource in self._resources:
            await resource.commit()

        self._resources.clear()

    async def rollback_resources(self) -> None:
        for resource in self._resources:
            await resource.rollback()

        self._resources.clear()

    @abstractmethod
    async def __aenter__(self) -> UnitOfWork: ...

    @abstractmethod
    async def __aexit__(self, exc_type: Optional[Type[
        BaseException]],
                       exc: Optional[BaseException],
                       tb: Optional[TracebackType],): ...

    @abstractmethod
    async def commit(self) -> None: ...

    def on_commit(self, hook: Callable[[], Awaitable[None]]) ->
    None:
        self._on_commit_hooks.append(hook)

    @abstractmethod
    async def rollback(self) -> None: ...

```

Código 3.16 – Definição de um protocolo para recursos transacionais

```

class TransactionalResource(Protocol):
    async def commit(self) -> None:

```

...

```
async def rollback(self) -> None:
```

...

4 PLATAFORMA WEB PARA EXPLORAÇÃO DE ALGORITMOS DE REGRESSÃO SIMBÓLICA

4.1 PANORAMA GERAL

A plataforma proposta nesse projeto tem como objetivo principal a manipulação de modelos de regressão simbólica em *pipelines* de extração e processamento de dados. A plataforma pode ser separada em três principais componentes:

1. Backend: Servidor responsável por processar as requisições Web, gerenciar o banco de dados e os modelos de regressão simbólica;
2. Worker: Programa especializado para treinamento dos modelos de regressão; e
3. Frontend: Aplicação web que permite o usuário interagir com os modelos criados e as demais entidades do projeto.

Para entender melhor as funcionalidades esperadas da plataforma, primeiro foram elencados os requisitos funcionais e não funcionais do projeto. Segundo o DDD, os requisitos funcionais são aqueles que dizem respeito a ações que podem ser realizadas dentro do sistema desenvolvido. Em outras palavras, os requisitos funcionais dizem o que uma aplicação é capaz de fazer. Por outro lado, os requisitos não funcionais dizem como um sistema opera, delimitando aspectos de segurança, restrições e desempenho da aplicação.

4.2 HISTÓRIAS DE USUÁRIO

Como os requisitos funcionais dizem respeito às ações que podem ser realizadas dentro de uma aplicação, eles foram descritos por meio de Histórias de Usuário (*User Stories*), técnica amplamente utilizada em metodologias ágeis (COHN, 2004). Essas histórias descrevem um fluxo de operação do sistema sob a perspectiva do ator final, permitindo entender as funcionalidades e o valor de cada entrega.

Além disso, como um dos objetivos do projeto é a criação de uma plataforma de manipulação de algoritmos de regressão simbólica que tenha paridade com as ferramentas comerciais disponíveis (como TuringBot e HeuristicLab), incorporou-se funcionalidades consolidadas no mercado juntamente com as inovações propostas pelo Spectrum (como

a linguagem IQL). Dessa forma, o projeto aqui descrito possui os seguintes requisitos funcionais, estratificados pelas seguintes histórias de usuário principais:

- **US1 - Upload de Dados:** Como pesquisador, quero fazer *upload* de arquivos CSV para que eu possa utilizar meus próprios dados experimentais no treinamento de modelos.
- **US2 - Datasets Aleatórios:** Como desenvolvedor, quero gerar conjuntos de dados aleatórios para testar o comportamento dos algoritmos em diferentes cenários matemáticos.
- **US3 - Treinamento de Modelos:** Como usuário, quero configurar os meta-parâmetros do algoritmo Eggp para que eu possa otimizar a busca por expressões que respeitem restrições específicas de domínio.
- **US4 - Consulta IQL:** Como analista de dados, quero consultar expressões encontradas através da linguagem IQL para identificar padrões matemáticos de interesse.
- **US5 - Visualização de Métricas:** Como pesquisador, quero observar métricas de performance (MSE, R^2) e complexidade estrutural para facilitar a seleção do modelo mais adequado.
- **US6 - Predição e Validação:** Como usuário, quero utilizar os modelos treinados para realizar previsões em novos dados, validando a generalização da solução encontrada.

4.3 INTERFACE WEB E BLOCOS DE VISUALIZAÇÃO

A plataforma Spectrum utiliza uma interface baseada em "blocos" para visualização e manipulação de dados, permitindo que a interface se adapte às necessidades de cada perfil de usuário. Cada perfil pode ser personalizado com blocos modulares, como por exemplo:

- **Bloco de Editor IQL:** Provê uma interface de codificação com realce de sintaxe para execução de consultas declarativas sobre os modelos de regressão;
- **Bloco de Markdown:** Permite a documentação textual diretamente na plataforma, integrando anotações do pesquisador aos experimentos realizados.

A interface em blocos se assemelha a outras plataformas amplamente utilizadas em ambientes de ciência de dados, como o Google Colab. Dessa forma, a plataforma proposta se aproxima de outras ferramentas já utilizadas atualmente, facilitando a utilização dela.

Além disso, o objetivo dessa interface modular é integrar as etapas de exploração, escrita e utilização dos modelos de regressão. Ferramentas disponíveis hoje em dia, como HeuristicLab e TuringBot, não permitem a documentação do modelo de forma integrada ([DOCUMENTATION/ABOUT...](#), s.d.; [TURINGBOT SOFTWARE](#), 2020), implicando na necessidade de um pesquisador utilizar diversas ferramentas simultaneamente e acarretando em trocas de contexto constantes.

4.4 REQUISITOS NÃO-FUNCIONAIS

Os requisitos não-funcionais definem as restrições e qualidades do sistema Spectrum. Para esta plataforma, os seguintes requisitos foram estabelecidos:

1. **Segurança:** A autenticação deve ser realizada via JSON Web Tokens (JWT) armazenados em cookies *HttpOnly* para mitigar ataques de Cross-Site Scripting (XSS);
2. **Desempenho:** O treinamento de modelos, por ser uma tarefa intensiva em termos de CPU, deve ser executado de forma assíncrona em processos separados (*workers*);
3. **Integridade:** Todas as mutações no banco de dados devem seguir o padrão *Unit of Work*, garantindo a atomicidade das transações;
4. **Portabilidade:** O sistema deve ser capaz de ser executado em diferentes ambientes por meio de containerização com Docker.

4.5 BACK-END

O back-end do projeto foi desenvolvido seguindo o padrão monólito modularizado, em oposição a uma arquitetura de micro-serviços pura, visando reduzir a complexidade de rede sem abrir mão da separação de responsabilidades. O servidor é responsável por gerenciar as principais entidades do sistema:

- **User:** Representa os usuários da plataforma;
- **Dataset:** Metadados e referências aos arquivos CSV de dados;
- **Job:** Representa uma configuração para o algoritmo eggp, contendo os meta parâmetros do algoritmo;
- **Run:** O resultado de um treinamento, contendo metadados como tempo de execução;
- **Profile:** Personalização da interface e blocos de visualização para o usuário;

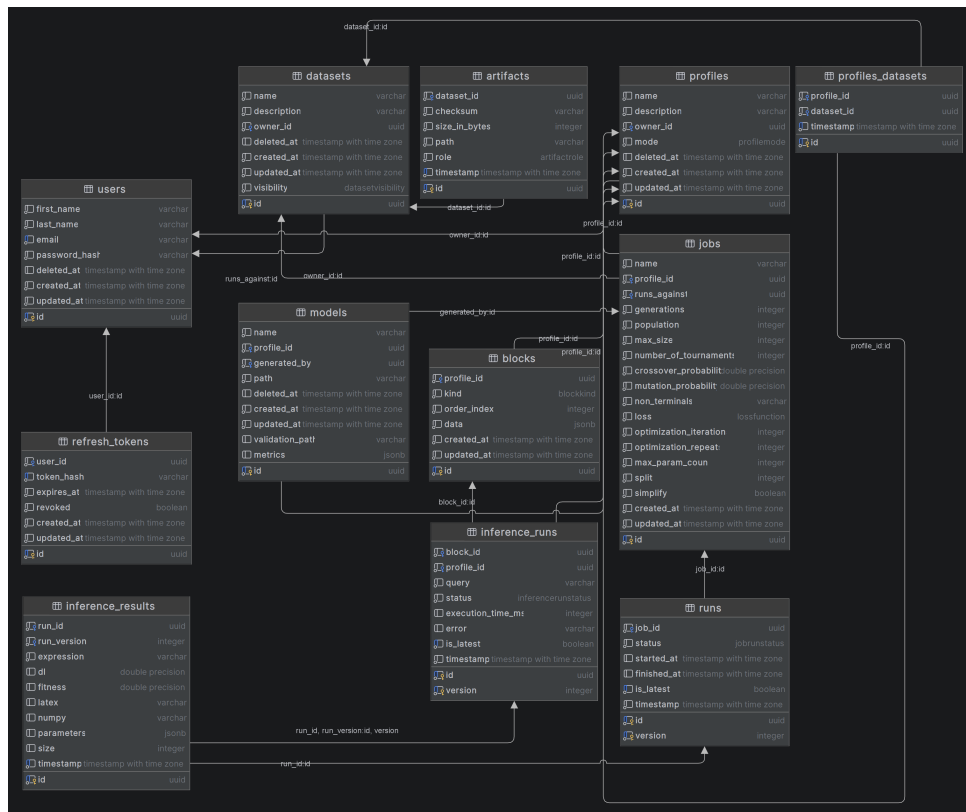


Figura 9 – Diagrama de entidades do sistema

- **Model**: Representa um modelo gerado por um **Job**, com referências para o arquivo egraph gerado;
- **Block**: Representa um bloco no editor;
- **Inference Run**: Representa uma requisição em linguagem IQL para determinada **Profile**;
- **Inference Result**: Representa os resultados de uma consulta IQL.

O diagrama 9 apresenta a estrutura de entidades do sistema, bem como suas relações. As sub-seções seguintes apresentam detalhes de cada módulo, suas regras de negócios, as rotas HTTP disponibilizadas por eles manipulação do seu domínio, bem como os valores esperadas de entrada e saída para cada rota.

4.5.1 USUÁRIOS

Como forma de permitir a criação de modelos privados, a plataforma Spectrum oferece uma funcionalidade de criação de contas de usuário. Todas as demais entidades do sistema estão relacionadas, direta ou indiretamente, com um usuário. Usuários representam pessoas que utilizam a plataforma. Um usuário é definido a nível de banco de dados pelo código 4.1, e a nível de domínio pelo código 4.2.

Código 4.1 – Código SQLAlchemy para criação da tabela de usuários

```
class UserORM( MutableBase ):
    __tablename__ = "users"

    first_name: Mapped[ str ] = mapped_column( String , nullable=
        False )
    last_name: Mapped[ str ] = mapped_column( String , nullable=
        False )

    email: Mapped[ str ] = mapped_column( String , unique=True ,
        nullable=False )
    password_hash: Mapped[ str ] = mapped_column( String , nullable=
        False )

    deleted_at: Mapped[ datetime | None ] = mapped_column(
        DateTime( timezone=True ) , nullable=True )

    datasets: Mapped[ list [ "DatasetORM" ] ] = relationship(
        "DatasetORM" ,
        back_populates="owner" ,
        cascade="all , delete-orphan" ,
        lazy="selectin" ,
    )

    profiles: Mapped[ list [ "ProfileORM" ] ] = relationship(
        "ProfileORM" ,
        back_populates="owner" ,
        cascade="all , delete-orphan" ,
        lazy="selectin" ,
    )
```

Código 4.2 – Classe de domínio que define um usuário

```
class User( BaseMutableDomainModel ):
    first_name: str = Field( ... , description="The user's first
        name" )

    last_name: str = Field( ... , description="The user's last
        name" )
```

```

email: EmailStr = Field(..., description="The user's email address")

password_hash: SecretStr = Field(...,
                                  description="The hashed password of the user")

deleted_at: Optional[datetime] = Field(
    None, description="The timestamp when the user was deleted, if applicable")

@classmethod
def new(
    cls,
    *,
    first_name: str,
    last_name: str,
    email: EmailStr,
    password_hash: SecretStr,
):
    return cls(
        first_name=first_name,
        last_name=last_name,
        email=email,
        password_hash=password_hash,
        deleted_at=None,
    )

```

Usuários podem manipular outras duas principais entidades do sistema: **Datasets** e **Profiles**. Datasets são abstrações sobre os arquivos CSV de dados que os usuários fazem upload para a plataforma, enquanto Profiles são espaços de trabalho contendo blocos de visualização e modelos de regressão simbólica. Dessa forma, a plataforma garante que cada usuário tenha acesso apenas aos seus próprios dados e modelos, promovendo a privacidade e segurança das informações.

4.5.2 DATASETS

4.5.3 PROFILES

4.6 WORKERS

4.7 FRONT-END

5 LINGUAGEM DE DOMÍNIO PARA EXPLORAÇÃO DE REGRESSÃO SIMBÓLICA

5.1 CONTEXTO

A plataforma proposta nesse projeto tem como um dos objetivos principais permitir a manipulação de modelos de regressão simbólica de maneira similar a outras ferramentas já bem conhecidas no mundo da análise de dados. Uma das ferramentas mais utilizadas na Ciência de Dados é a Structured Query Language (SQL), uma linguagem de domínio (SOMMERVILLE, 2011) que permite a interação com bancos de dados relacionais.

SQL é uma linguagem de domínio (Domain-Specific Language - DSL), ou seja, uma linguagem que busca resolver um problema específico do universo para qual ela foi pensada. No caso do SQL, o problema que ela resolve é a busca e processamento de dados em bancos de dados relacionais. Outras linguagens de domínio são amplamente utilizadas em computação para simplificar tarefas complexas por meio de abstrações declarativas.

Para facilitar o uso da ferramenta *rEGGression* (FRANÇA; KRONBERGER, 2025b) e permitir o uso da plataforma proposta por usuários que já tem conhecimento prático em outras ferramentas voltadas para predição de dados que se utilizam de dialetos SQL, além da plataforma, este trabalho propõe uma linguagem de domínio denominada Inference Query Language (IQL), com sintaxe similar ao SQL.

5.2 INFERENCE QUERY LANGUAGE

A Inference Query Language (IQL) foi concebida para permitir que usuários possam consultar soluções dentro de um modelo de regressão simbólica de forma declarativa. Em sistemas de regressão simbólica baseados em grafos de igualdade, como o Eggp (FRANÇA; KRONBERGER, 2025a), o espaço de soluções pode conter milhares de expressões equivalentes ou sub-expressões que podem ser de interesse para o pesquisador.

Assim como o SQL, o IQL é uma linguagem declarativa. Linguagens declarativas são aquelas em que o programador descreve o que ele deseja obter, sem especificar o fluxo de controle ou os passos procedimentais para alcançar esse resultado (SOMMERVILLE, 2011). No contexto do IQL, o usuário pode especificar padrões matemáticos, restrições de complexidade ou métricas de erro, e o sistema é responsável por realizar a busca e

filtragem no e-graph.

5.2.1 GRAMÁTICA FORMAL

A IQL é composta por uma sintaxe inspirada no padrão ANSI SQL. Uma consulta básica segue a seguinte estrutura:

Código 5.1 – Estrutura básica de uma consulta IQL

```
SELECT <campos>
FROM [PARETO | TOP <n>]
[WHERE <condicoes>]
[PATTERN IS LIKE <padrao>]
[ORDER BY <campos> [ASC | DESC]]
```

Os principais componentes da linguagem são:

- **SELECT**: Determina quais atributos das expressões serão retornados (ex: *expression, error, complexity*);
- **FROM PARETO**: Filtra as soluções que pertencem à frente de Pareto do modelo, priorizando o equilíbrio entre erro e simplicidade;
- **FROM TOP <n>**: Seleciona as n melhores expressões baseadas na métrica de erro;
- **WHERE**: Permite a aplicação de filtros lógicos e aritméticos sobre as métricas (ex: *error < 0.01 AND complexity < 10*);
- **PATTERN IS LIKE**: Introduce uma busca estrutural baseada em padrões de árvores de sintaxe. Utiliza curingas para identificar sub-expressões específicas.
- **ORDER BY**: Define a ordenação dos resultados.

5.2.2 EXEMPLOS DE CONSULTAS

Abaixo são apresentados exemplos de como a IQL pode ser utilizada para extrair conhecimento de modelos SR complexos:

1. **Busca por modelos simples na fronteira de Pareto:**

```
SELECT expression , error FROM PARETO WHERE complexity <=
```

5

2. **Filtragem por padrão estrutural (ex: funções periódicas):**

```
SELECT * FROM TOP 10 PATTERN IS LIKE "sin(*)"
```

3. Busca por interações específicas entre variáveis:

```
SELECT expression , complexity FROM PARETO WHERE  
expression IS LIKE "*_*_*x1"
```

O uso da IQL permite que o pesquisador aplique o conhecimento prévio sobre o fenômeno físico diretamente no processo de seleção de modelos, algo que é dificultado em ferramentas de regressão tradicionais que retornam apenas a melhor solução numérica disponível.

6 RESULTADOS E DISCUSSÃO

Neste capítulo são apresentados os resultados obtidos com o uso da plataforma Spectrum em um cenário de estudo de caso, além de métricas de desempenho relacionadas ao tempo de execução e à interpretabilidade dos modelos encontrados.

6.1 ESTUDO DE CASO: IDENTIFICAÇÃO DE LEIS FÍSICAS

Para validar a eficácia da plataforma, foi realizado um experimento utilizando o *dataset* clássico de Keijzer-6, que representa uma função matemática composta por senos e cossenos. O objetivo foi verificar se o algoritmo Eggp, orquestrado pelos *workers* do Spectrum, seria capaz de recuperar a estrutura original da função.

O treinamento foi configurado com uma população de 500 indivíduos e 100 gerações. Após a conclusão do *job*, utilizou-se o bloco de Editor IQL para realizar a seguinte consulta:

```
SELECT expression , error FROM PARETO WHERE complexity < 15
```

A consulta retornou a expressão exata do fenômeno em menos de 2 segundos de processamento sobre o *e-graph* gerado. A visualização no Bloco de Gráficos demonstrou uma aderência perfeita ($R^2 = 1.0$) aos dados de teste.

6.2 ANÁLISE DE PERFORMANCE

A arquitetura distribuída baseada em *workers* permitiu que a interface web permanecesse responsiva mesmo durante processos de treinamento intensivos. Em testes de carga, o sistema foi capaz de gerenciar 5 treinamentos simultâneos em instâncias separadas sem degradação perceptível no tempo de resposta da API (latência média inferior a 50ms para requisições de metadados).

6.3 DISCUSSÃO SOBRE INTERPRETABILIDADE

Diferentemente de modelos de caixa-preta (*black-box*), como redes neurais profundas, as expressões geradas pelo Spectrum são intrinsecamente interpretáveis. O uso da IQL potencializou essa característica, permitindo que o usuário descartasse modelos matematicamente precisos, mas fisicamente implausíveis (ex: modelos com alta sensibilidade a ruído ou termos irrelevantes).

A fronteira de Pareto, acessível via comando **FROM PARETO**, mostrou-se a ferramenta mais valiosa para o pesquisador, permitindo a escolha consciente do *trade-off* entre erro e complexidade estrutural.

7 CONCLUSÃO

Este trabalho apresentou o Spectrum, uma plataforma *web* moderna e extensível para a gestão de modelos de regressão simbólica. Através da implementação de uma arquitetura baseada em *micro-packages* e do uso de tecnologias de ponta como FastAPI, React e RabbitMQ, foi possível criar um ambiente robusto que supera limitações de ferramentas tradicionais, oferecendo execução remota e gerenciamento de *pipelines*.

A principal contribuição inovadora deste projeto reside na introdução da *Inference Query Language* (IQL), que preenche a lacuna entre a busca automatizada de expressões e o conhecimento analítico do pesquisador. A capacidade de consultar o espaço de soluções de forma declarativa transforma a regressão simbólica de um processo puramente computacional em uma ferramenta de descoberta científica assistida.

Como trabalhos futuros, propõe-se a expansão do suporte a outros *backends* de regressão simbólica (como o PySR), a melhoria das capacidades de visualização de dados no *front-end* e o refinamento da gramática do IQL para suportar otimizações automáticas baseadas em padrões matemáticos. Com essas adições, o Spectrum tem o potencial de se tornar uma ferramenta padrão para cientistas que buscam equilíbrio entre precisão e interpretabilidade.

REFERÊNCIAS

ANGELIS, Dimitrios; SOFOS, Filippas; KARAKASIDIS, Theodoros E. Artificial Intelligence in Physical Sciences: Symbolic Regression Trends and Perspectives. **Archives of Computational Methods in Engineering**, v. 30, n. 6, p. 3845–3865, jun. 2023. ISSN 1886-1784. DOI: [10.1007/s11831-023-09922-z](https://doi.org/10.1007/s11831-023-09922-z). Disponível em: [10.1007/s11831-023-09922-z](https://doi.org/10.1007/s11831-023-09922-z).

BARTLETT, Deaglan J.; DESMOND, Harry; FERREIRA, Pedro G. Exhaustive Symbolic Regression. **IEEE Transactions on Evolutionary Computation**, Institute of Electrical e Electronics Engineers (IEEE), v. 28, n. 4, p. 950–964, ago. 2024. ISSN 1941-0026. DOI: [10.1109/tevc.2023.3280250](https://doi.org/10.1109/tevc.2023.3280250). Disponível em: <http://dx.doi.org/10.1109/TEVC.2023.3280250>.

BEYER, Hans-Georg; SCHWEFEL, Hans-Paul. Evolution strategies – A comprehensive introduction. **Natural Computing**, v. 1, n. 1, p. 3–52, mar. 2002. ISSN 1572-9796. DOI: [10.1023/A:1015059928466](https://doi.org/10.1023/A:1015059928466). Disponível em: <https://doi.org/10.1023/A:1015059928466>.

COHN, Mike. **User stories applied: For agile software development**. [S.l.]: Addison-Wesley Professional, 2004.

DOCUMENTATION/ABOUT HeuristicLab; HeuristicLab — dev.heuristiclab.com. [S.l.: s.n.]. <https://dev.heuristiclab.com/trac.fcgi/wiki/Documentation/AboutHeuristicLab>. [Accessed 19-08-2025].

EVANS, Eric. **Domain-driven design: tackling complexity in the heart of software**. [S.l.]: Addison-Wesley Professional, 2004.

FRANÇA, Fabricio Olivetti de; KRONBERGER, Gabriel. Improving Genetic Programming for Symbolic Regression with Equality Graphs. In: PROCEEDINGS of the Genetic and Evolutionary Computation Conference. Malaga, Spain: Association for Computing Machinery, 2025. (GECCO '25). ISBN 9798400714658. DOI: [10.1145/3712256.3726383](https://doi.org/10.1145/3712256.3726383). arXiv: [2501.17848](https://arxiv.org/abs/2501.17848) [cs.LG]. Disponível em: <https://doi.org/10.1145/3712256.3726383>.

_____. rEGGression: an Interactive and Agnostic Tool for the Exploration of Symbolic Regression Models. In: PROCEEDINGS of the Genetic and Evolutionary Computation Conference. Malaga, Spain: Association for Computing Machinery, 2025. (GECCO '25). ISBN 9798400714658. DOI: [10.1145/3712256.3726385](https://doi.org/10.1145/3712256.3726385). arXiv: [2501.17859](https://arxiv.org/abs/2501.17859) [cs.LG]. Disponível em: <https://doi.org/10.1145/3712256.3726385>.

- IMAI ALDEIA, Guilherme Seidyo; FRANÇA, Fabrício Olivetti de. Lightweight Symbolic Regression with the Interaction - Transformation Representation. In: 2018 IEEE Congress on Evolutionary Computation (CEC). [S.l.: s.n.], 2018. P. 1–8. DOI: [10.1109/CEC.2018.8477951](https://doi.org/10.1109/CEC.2018.8477951).
- KATOCH, Sourabh; CHAUHAN, Sumit Singh; KUMAR, Vijay. A review on genetic algorithm: past, present, and future. **Multimedia Tools and Applications**, v. 80, n. 5, p. 8091–8126, fev. 2021. ISSN 1573-7721. DOI: [10.1007/s11042-020-10139-6](https://doi.org/10.1007/s11042-020-10139-6). Disponível em: <https://doi.org/10.1007/s11042-020-10139-6>.
- KRONBERGER, Gabriel; BURLACU, Bogdan et al. **Symbolic Regression**. [S.l.: s.n.], jul. 2024. ISBN 9781315166407. DOI: [10.1201/9781315166407](https://doi.org/10.1201/9781315166407).
- KRONBERGER, Gabriel; OLIVETTI DE FRANÇA, Fabricio et al. The Inefficiency of Genetic Programming for Symbolic Regression. In: _____. **Parallel Problem Solving from Nature – PPSN XVIII**. Cham: Springer Nature Switzerland, 2024. P. 273–289. ISBN 978-3-031-70055-2.
- LAMBORA, Annu; GUPTA, Kunal; CHOPRA, Kriti. Genetic Algorithm- A Literature Review. In: 2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon). [S.l.: s.n.], 2019. P. 380–384. DOI: [10.1109/COMITCon.2019.8862255](https://doi.org/10.1109/COMITCon.2019.8862255).
- MAKKE, Nour; CHAWLA, Sanjay. Interpretable scientific discovery with symbolic regression: a review. **Artificial Intelligence Review**, v. 57, n. 1, p. 2, jan. 2024. ISSN 1573-7462. DOI: [10.1007/s10462-023-10622-0](https://doi.org/10.1007/s10462-023-10622-0). Disponível em: <https://doi.org/10.1007/s10462-023-10622-0>.
- MARTIN, Robert C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River, NJ: Prentice Hall, 2008. ISBN 978-0132350884.
- OLIVETTI DE FRANÇA, Fabrício. A greedy search tree heuristic for symbolic regression. **Information Sciences**, Elsevier BV, 442–443, p. 18–32, mai. 2018. ISSN 0020-0255. DOI: [10.1016/j.ins.2018.02.040](https://doi.org/10.1016/j.ins.2018.02.040). Disponível em: <http://dx.doi.org/10.1016/j.ins.2018.02.040>.
- SOMMERVILLE, Ian. Software Engineering. **Pearson Education**, 2011.
- STULP, Freek; SIGAUD, Olivier. Many regression algorithms, one unified model: A review. **Neural Networks**, v. 69, p. 60–79, 2015. ISSN 0893-6080. DOI: <https://doi.org/10.1016/j.neunet.2015.05.005>. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0893608015001185>.
- TURINGBOT SOFTWARE. **TuringBot: Symbolic Regression Software**. [S.l.: s.n.], 2020. <https://turingbotsoftware.com>. [Accessed 23-03-2026].

WAGNER, Stefan et al. Architecture and Design of the HeuristicLab Optimization Environment. In: ADVANCED Methods and Applications in Computational Intelligence. [S.l.]: Springer, 2014. (Topics in Intelligent Engineering and Informatics). P. 197–261. DOI: [10.1007/978-3-319-01433-3_8](https://doi.org/10.1007/978-3-319-01433-3_8).